CSCI 6461 Computer System Architecture

Project Part 3

Operational Manual

Team 3

1. System Overview

The Part 3 simulator is a Java Swing application that emulates the 16-bit CISC machine defined in the course ISA. It loads a pre-assembled object file (load.txt) into a 2048-word main memory, fetches and executes instructions one at a time, and presents all registers, the cache, and an interactive console on a single window.

Part 3 adds three features on top of Part 2:

- A stand-alone two-pass assembler that converts source.txt into listing.txt and load.txt.
- An I/O subsystem with keyboard and printer buffers that talk to the Console tab.
- Program 2, which loads a paragraph of English text from paragraph.txt and lets the operator search it for a word.

System Requirements

- Java JDK 17 or later (the supplied JAR was built with JDK 17).
- simulator.jar, load.txt and paragraph.txt must be together in the same folder.
- Run with: java -jar simulator.jar

2. How to Run

From the JAR file

1. Place simulator.jar, load.txt and paragraph.txt in the same folder.
2. Open a terminal and change to that folder.
3. Run: java -jar simulator.jar
4. The simulator window opens.

From source code

5. Change to the project root (the folder that contains src/).
6. Compile every .java file into out/:

```
find src -name "*.java" | xargs javac -d out
```

7. Run the front-panel entry point:

```
java -cp out front.Main
```

Running the assembler (optional)

Assembler.java produces load.txt and listing.txt from a source file written in the course assembly language. From the folder that contains Assembler.class:

```
java Assembler source.txt listing.txt load.txt
```

The first argument is the source file (default source.txt), the second is the listing (default listing.txt), and the third is the load file consumed by IPL (default load.txt). Labels and decimal operands are translated into six-digit octal addresses and words. Place the generated load.txt next to simulator.jar before clicking IPL.

3. Console Layout

The window has four regions: a register column on the left, a tabbed console in the centre, a cache display, and a row of operator buttons along the bottom.

Left column - Registers

Each register is shown as a label, an editable text field (octal), and a LOAD button. Registers are grouped into four titled sections:

Section	Registers
General Purpose Registers	R0, R1, R2, R3 (16 bits each)
Index Registers	X1, X2, X3 (16 bits each)
Address	MAR, MBR, MFR, CC
Other Registers	MSR, IR, PC (PC is 12 bits)

Centre - Console tab

A scrollable text area that serves as both the printer display and the keyboard input device. The simulator logs each instruction as PC and its 16-bit binary. Program 2 output appears here, and text typed while the program is running becomes keyboard input for the IN instruction.

Cache panel

A 16-row two-column table (Tag, Data) showing the current contents of the FIFO cache. The table is refreshed after every instruction.

Bottom row - Buttons

Button	Action
IPL	Initial Program Load. Reads load.txt into memory, reads paragraph.txt into memory starting at address 100 (decimal), sets PC to 10 and R3 to PC+1, and enables SS, Run, and Search.
SS	Single Step. Executes one instruction from the current PC and refreshes every panel.
Run	Runs continuously on a background thread. Stops at HLT, at a zero word, or after 5000 steps.
Halt	Stops continuous execution.
Reset	Clears every register and the cache, re-enables IPL.
Store	Writes the value currently typed into the selected register field into that register (octal).
Search	Runs Program 2. Reads the paragraph out of memory, prints every sentence, prompts for a search word, and reports all matches.
LOAD	Per-register. Same behaviour as Store, but tied to the row it sits next to.

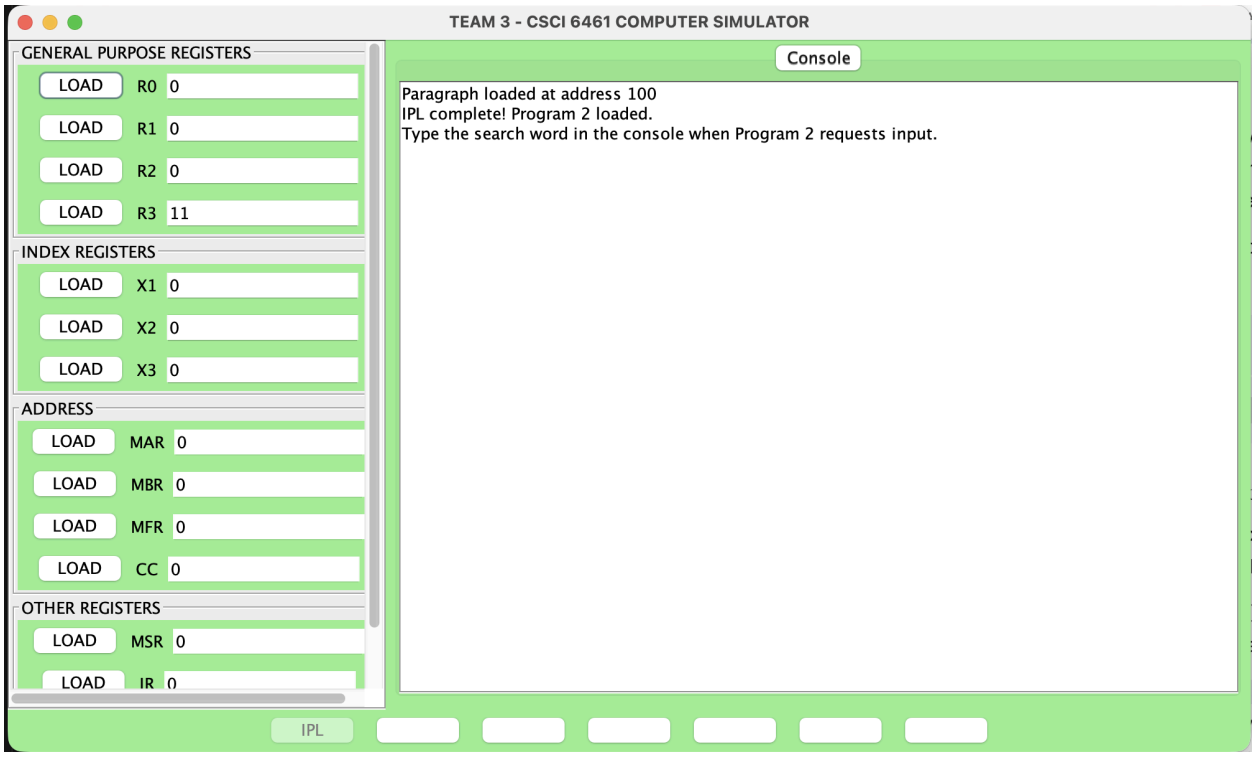
4. Step-by-Step Procedures

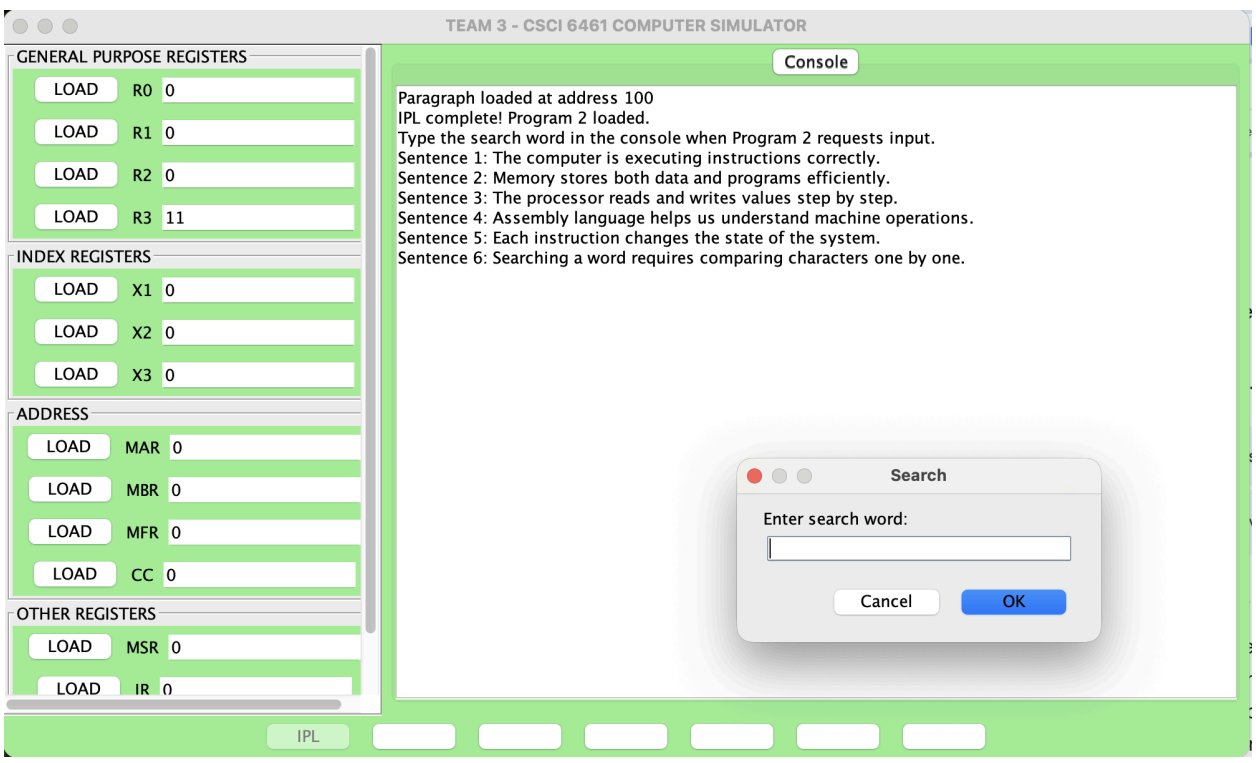
4.1 Running Program 2 - Paragraph Search

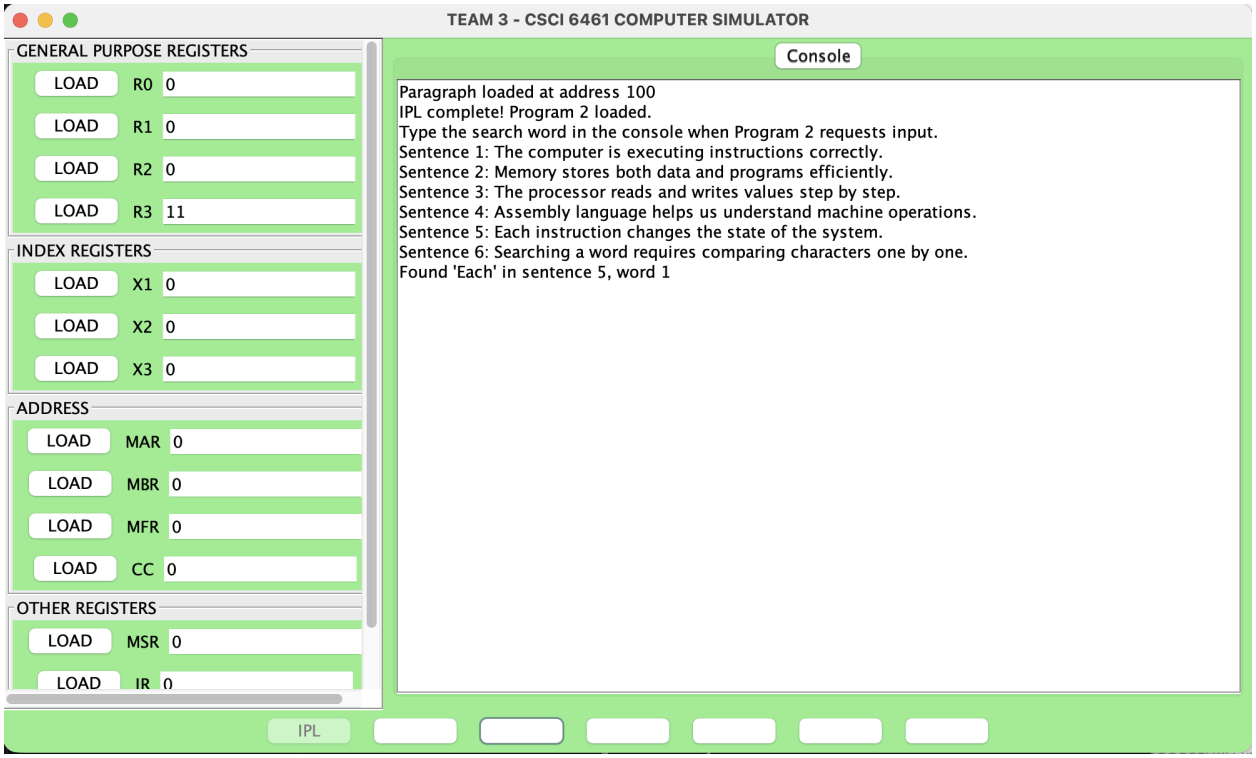
1. Click IPL. The paragraph from paragraph.txt is loaded into memory starting at address 100.
2. Click Search.
3. The console lists every sentence in the paragraph, numbered 1, 2, 3, ...
4. A dialog box asks for the search word. Type a word and click OK.
5. For every occurrence the console prints: Found <word> in sentence N, word M.
6. If the word is not present, the console prints: '<word>' not found in paragraph.

Example output

```
Sentence 1: The computer is executing instructions correctly.
Sentence 2: Memory stores both data and programs efficiently.
Sentence 3: The processor reads and writes values step by step.
...
Found 'memory' in sentence 2, word 1
```


TEAM 3 - CSCI 6461 COMPUTER SIMULATOR
GENERAL PURPOSE REGISTERS
LOAD R0 0
LOAD R1 0
LOAD R2 0
LOAD R3 11
INDEX REGISTERS
LOAD X1 0
LOAD X2 0
LOAD X3 0
ADDRESS
LOAD MAR 0
LOAD MBR 0
LOAD MFR 0
LOAD CC 0
OTHER REGISTERS
LOAD MSR 0
LOAD IR 0
Console
Paragraph loaded at address 100
IPL complete! Program 2 loaded.
Type the search word in the console when Program 2 requests input.
IPL


TEAM 3 - CSCI 6461 COMPUTER SIMULATOR
GENERAL PURPOSE REGISTERS
LOAD R0 0
LOAD R1 0
LOAD R2 0
LOAD R3 11
INDEX REGISTERS
LOAD X1 0
LOAD X2 0
LOAD X3 0
ADDRESS
LOAD MAR 0
LOAD MBR 0
LOAD MFR 0
LOAD CC 0
OTHER REGISTERS
LOAD MSR 0
LOAD IR 0
Console
Paragraph loaded at address 100
IPL complete! Program 2 loaded.
Type the search word in the console when Program 2 requests input.
Sentence 1: The computer is executing instructions correctly.
Sentence 2: Memory stores both data and programs efficiently.
Sentence 3: The processor reads and writes values step by step.
Sentence 4: Assembly language helps us understand machine operations.
Sentence 5: Each instruction changes the state of the system.
Sentence 6: Searching a word requires comparing characters one by one.
Search
Enter search word:
Cancel
OK
IPL



4.3 Setting a register manually

8. Click the text field next to the register, for example PC.
9. Type the value in octal.
10. Click LOAD on that row, or click the global Store button.
11. The console confirms: 'Stored value <v> in <register>.'

4.4 Verifying the cache

12. Click IPL.
13. Click SS once. A new row (tag, data) appears in the cache table. The terminal prints CACHE MISS.
14. Click SS again, re-reading the same address. No new row is added and the terminal prints CACHE HIT.
15. Keep stepping until all 16 rows are filled.
16. The 17th unique address evicts the oldest row (FIFO).

4.5 Using keyboard input

If the running program executes an IN instruction and the keyboard buffer is empty, the simulator stalls - it will keep trying the same IN. Click inside the Console tab and type one or more characters; as soon as a character is typed, IN consumes it and execution resumes.

5. Load File Format

load.txt is a plain text file with one memory word per line. Each line has two fields separated by whitespace:

```
000016 102124
000017 026002
000020 002100
```

- First column: six-digit octal memory address.
- Second column: six-digit octal 16-bit word to store at that address.
- Blank lines and lines beginning with ';' are ignored as comments.
- The file must sit in the same folder as simulator.jar.

6. Paragraph File Format

paragraph.txt contains plain English text, one or more sentences per line. Sentence terminators (. ! ?) separate sentences when Program 2 tokenises. A sample file:

```
The computer is executing instructions correctly.
Memory stores both data and programs efficiently.
The processor reads and writes values step by step.
```

The loader concatenates the lines with spaces, appends a '#' sentinel, stores each character as its ASCII code into consecutive memory words starting at address 100, and records them in the cache.

7. Error Messages

Message	Meaning
Failed to load program file: ...	load.txt or paragraph.txt is missing or unreadable. Put the files next to the JAR and click IPL again.
HLT reached. Stopping.	Normal end of program - the fetched word was all zeros.
SS error: ...	An exception escaped during single-step execution. Click Reset, then IPL to restart.
Run error: ...	An exception escaped during continuous execution. Click Reset, then IPL to restart.
No paragraph found at address 100.	Search was clicked but paragraph.txt was not loaded. Click IPL first.
Search cancelled or empty input.	The search dialog was dismissed or left blank.
'<word>' not found in paragraph.	The word was not present in any sentence.
Invalid input. Please enter a valid octal value.	A register field contained a non-octal value when Store was clicked.

Message	Meaning
Fault dialog box	A machine fault occurred. MFR shows the code. Click OK to dismiss; execution resumes from memory[1].

8. Files

File	Purpose
simulator.jar	Runnable JAR containing all compiled classes. Entry point front.Main.
load.txt	Pre-assembled object file loaded at IPL.
paragraph.txt	Text paragraph used by Program 2.
source.txt	Assembly source used to produce load.txt.
listing.txt	Listing produced by the assembler for reference.
manifest.txt	Manifest used when building the JAR.